



@DevWithRishabh

FROM BASICS TO ADVANCED



Declarative
UI Library

- ✓ Beginner Friendly
- ✓ Clear Examples
- ✓ Handwritten Style
- ✓ Interview Ready
- ✓ Project Focused



COMPLETE REACT NOTES

Components

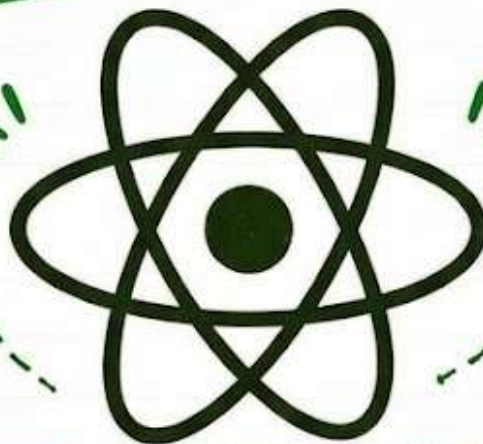


Learn Once
Use Everywhere

Hooks

useState()
useEffect()
useRef()
useContext()
...and more!

```
import React from 'react';  
  
function App() {  
  return (  
    <div className="app">  
      <h1>Hello, React! </h1>  
    </div>  
  );  
}  
  
export default App;
```



Build Amazing
Web Apps



Components
Driven



Reusable
Code



Fast &
Efficient

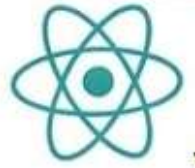


Developer
Friendly

Your Complete Guide to Master React



</> DevWithRishabh </>



1. REACT INTRODUCTION & JSX

The Basics of Modern UI Development

1 WHAT IS REACT?

- React is a JavaScript library for building user interfaces.
- Developed and maintained by Facebook (Meta).
- Component-based, declarative and efficient.
- Used to build single-page applications and complex UIs.



React is all about building reusable UI components.

2 KEY FEATURES

- Component-Based
- Declarative
- Learn Once, Write Anywhere
- Virtual DOM
- Unidirectional Data Flow
- Huge Ecosystem

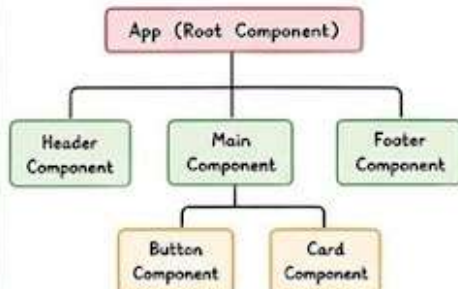
WHY REACT?

- Fast & Efficient
- Reusable Components
- Easy to Learn
- Great Performance
- Strong Community



3 COMPONENT-BASED ARCHITECTURE

UI is broken into small, independent and reusable pieces called components.



BENEFITS

- Reusable
- Maintainable
- Testable
- Scalable



4 WHAT IS JSX?

JSX (JavaScript XML) is a syntax extension for JavaScript. It allows us to write HTML-like code in JavaScript.



```

const element = <h2>Hello, React! </h2>;
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(element);
  
```



JSX Makes it easier to:

- Create UI structure
- Keep HTML and logic in the same place

5 JSX RULES

- JSX must return a single parent element.
- Close all tags (e.g. `<br />`, `<img />`).
- Use `className` instead of `class`.
- Use `htmlFor` instead of `for`.
- Use `{}` to embed JavaScript expressions.
- Comments in JSX: `{/* This is a comment */}`

EXAMPLE

```

const element = (
  <div className="box">
    <h1>Hello, React! </h1>
    <p>This is JSX </p>
  </div>
);
  
```

Single parent

6 JSX vs HTML

Feature	HTML	JSX
Attribute Naming	class, for	className, htmlFor
Return Multiple Elements	Yes	No (use single parent)
Self Closing Tags	Optional	Required <code>
</code>
JS in Markup	Not Allowed	Allowed with <code>{}</code>
Comments	<code><!-- comment --></code>	<code>{/* comment */}</code>

7 MINIMAL REACT APP EXAMPLE

```

// App.js
function App() {
  return (
    <div className="app">
      <h1>Welcome to React! </h1>
      <p>Let's build something awesome.</p>
    </div>
  );
}
export default App;
  
```

JS

```

// index.js
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(
  document.getElementById('root')
);
root.render(<App />);
  
```

index.js



Everything in React starts from a root component rendered into the DOM.

8 REACT APP STRUCTURE (BASIC)



my-react-app/	
├─ public/	→ Static files (HTML template)
│ └─ index.html	→ Root HTML file
└─ src/	
└─ App.js	→ Main App component
└─ index.js	→ Entry point of React app
└─ index.sss	→ Global styles
package.json	→ Project config & dependencies



React + JSX = Powerful way to build fast, interactive and scalable user interfaces!



2. COMPONENTS & PROPS

</> DevWithRishabh </>



1 WHAT IS A COMPONENT?

- Components are independent, reusable pieces of UI.
- They can be function or class based.
- Every component must return some JSX.
- Components can be composed together to build complex UIs.



Build UIs like blocks!



Think of components as JavaScript functions that return HTML (JSX).

2 TYPES OF COMPONENTS

FUNCTION COMPONENT

```
function Welcome() {
  return <div>Hello, React! </div>;
}
```

- Simple & concise
- Uses Hooks (useState, useEffect, etc.)
- Preferred in modern React

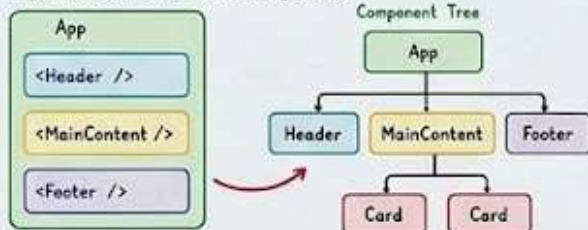
CLASS COMPONENT

```
class Welcome extends React.Component {
  render() {
    return <div>Hello, React! </div>;
  }
}
```

- Uses ES6 class syntax
- Has its own state & lifecycle methods
- Used in legacy codebases

3 COMPONENT COMPOSITION

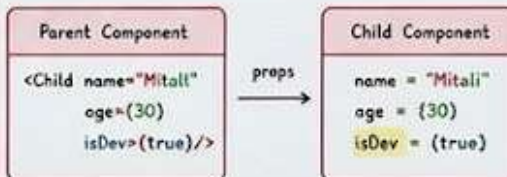
Components can render other components.



Break UI into small components. Compose them to build big features.

4 PROPS (PROPERTIES)

- Props (short for properties) are used to pass data from parent component to child component.
- Props are read-only in the child component.
- They help make components dynamic and reusable.



WHY PROPS?

- Pass data
- Make components reusable
- Keep code DRY

5 EXAMPLE: PASSING PROPS

Parent.js

```
import UserCard from './UserCard';

function App() {
  return (
    <div>
      <Header>
      <MainContent>
      <Footer>
    </div>
  );
}

export default App;
```

UserCard.js

```
function UserCard(props) {
  return (
    <div className='card'>
      <div>{props.name}</div>
      <div>{props.age}</div>
    </div>
  );
}

export default UserCard;
```



Props flow in one direction: Parent → Child

6 DESTRUCTURING PROPS

Without Destructuring

```
function UserCard(props) {
  return (
    <div>
      <div>{props.name}</div>
      <div>{props.age}</div>
    </div>
  );
}
```

With Destructuring

```
function UserCard({ name, age }) {
  return (
    <div>
      <div>{name}</div>
      <div>{age}</div>
    </div>
  );
}
```



Destructuring makes code cleaner and easier to read.

7 DEFAULT PROPS

```
function Button({ text = 'Click Me', color = 'blue' }) {
  return (
    <button className='button' background={color}>
      {text}
    </button>
  );
}
```

Usage

```

<Button /> // Click Me. Blue
<Button text='Submit' /> // Submit. Blue
<Button text='Test' color='green' /> // Save. Green
  
```



Default props are used when no value is passed.

8 CHILDREN PROP

We use JSX as children in a component using the special children prop.

```
function Card({ children }) {
  return (
    <div className='card'>
      {children}
    </div>
  );
}
```

Usage

```

<Card>
  <div>Hello</div>
</Card>
  
```

9 PROPS ARE READ-ONLY

- You cannot modify props in the child component.
- React will throw an error.
- If you need to change data, use state.

```
function Card({ name }) {
  name = 'Changed'; // ❌ Error!
  return <div>{name}</div>;
}
```

Props down, events up!
(Data down to child, actions up to parent)

Components make your UI reusable. Props make your components dynamic!

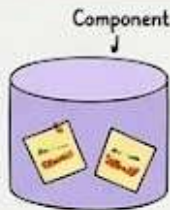
</> DevWithRishabh </>

3. STATE & useState

Manage Data That Changes Over Time

1 WHAT IS STATE?

- State is a built-in React object used to store data that can change over time.
- When state changes, the component re-renders automatically.
- State is local and private to the component.



Changes → Re-render



Use state for dynamic data: user input, toggles, counters, API data, etc.

2 THE useState HOOK

useState is a React Hook that lets you add data to function components.

```
// Import
import { useState } from 'react';

// Syntax
const [state, setState] = useState(initialValue);
```



Calling setState(newValue) updates the state and triggers a re-render.

SYNTAX BREAKDOWN

state → current value
setState → function to update the value
initialValue → initial state value

3 EXAMPLE: COUNTER APP

```
import { useState } from 'react';

function Counter() {
  const [DevWithRishabh, setDevWithRishabh] = useState(1);

  return (
    <div className="counter">
      <h3>Count: {count}</h3>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
      <button onClick={() => setCount(count - 1)}>
        Decrement
      </button>
    </div>
  );
}

export default DevWithRishabh;
```

OUTPUT



Click buttons →
State updates →
UI re-renders

4 UPDATING STATE

1. Set new value

```
setCount(10); // Replace with 10
setName('React'); // Replace with "React"
```

2. Update based on previous state

```
setCount(prev => prev + 1); // Safe way
```



Use the functional update when new state depends on the previous state.

BEST PRACTICES

- ✓ State is private
- ✓ Keep state minimal
- ✓ Don't mutate state
- ✓ Use meaningful state names

5 MULTIPLE STATE VARIABLES

You can use multiple useState calls.

```
const [name, setName] = useState('Nitali');
const [age, setAge] = useState(25);
const [isOnline, setIsOnline] = useState(false);
```

STATE EXAMPLE

State	Type	Initial Value
name	string	'Nitali'
age	number	25
isOnline	boolean	true

6 STATE WITH DIFFERENT DATA TYPES

String

“

```
const [name, setName] = useState('Nitali');
```

Number

123

```
const [count, setCount] = useState(0);
```

Boolean



```
const [isActive, setIsActive] = useState(false);
```

Array



```
const [items, setItems] = useState([]);
```

Object



```
const [user, setUser] = useState({});
```

7 IMPORTANT NOTES

- State updates are asynchronous.
- Do not rely on immediate state value after calling setState.
- Never mutate state directly.
- State is preserved between re-renders.
- Each component instance has its own state.



✗ Wrong (Mutating State)

```
const count = state.count + 1;
setState(state);
// May not re-render
```



✓ Correct Way

```
setCount(prev => ({
  ...prev,
  count: prev.count + 1
}));
```

8 COMMON PATTERNS

Toggle

```
const [on, setOn] = useState(false);
```

```
setOn(prev => !prev);
```



Form Input

```
const [text, setText] = useState('');
```

```
<input value={text}
  onChange={e =>
    setText(e.target.value)}
  />
```

Type here...

Reset State

```
const [count, setCount] = useState(0);
```

```
const reset = () =>
  setCount(0);
```



TIP: If the data can change and the UI should update, store it in state with useState!



4. EVENT HANDLING & FORMS

Handle Interactions. Collect Data. Build Amazing Apps! ❤️

1 EVENT HANDLING IN REACT

- ✓ React uses camelCase event names.
- ✓ Pass a function as the event handler.
- ✓ Do not call the function, pass the reference.
- ✓ Event handling works the same way in class & function components.



```
function Button() {
  const handleClick = () => {
    alert('Button Clicked! 🎉');
  };
  return <button onClick={handleClick}>Click Me</button>;
}
```

JS



Use arrow functions to avoid binding issues.

2 COMMON EVENTS

Category	Events	Description
Mouse Events	onClick, onDoubleClick, onMouseEnter, onMouseLeave, onMouseDown, onMouseUp	Handle mouse interactions
Keyboard Events	onKeyDown, onKeyUp, onKeyPress	Handle keyboard interactions
Form Events	onChange, onSubmit, onFocus, onBlur	Handle form interactions
Clipboard Events	onCopy, onPaste, onCut	Clipboard interactions
UI Events	onFocus, onStar, onScroll, onLoad	General UI interactions



React events are **SyntheticEvents** (cross-browser compatible).

3 PASSING ARGUMENTS TO EVENT HANDLERS

1. Arrow function

```
<button onClick={() => sayHi('Mukul')}>
  Say Hi
</button>
```

2. Function reference

```
const sayHi = (name) => {
  alert('Hi! ' + name);
};
<button onClick={() => sayHi('Mukul')}>
  Say Hi
</button>
```



Use arrow function when you need to pass arguments.

4 FORMS IN REACT

- ✓ In React, form elements are controlled by state.
- ✓ The input value is stored in state.
- ✓ Update state on every change.
- ✓ Handle form submission using onSubmit.



Always prevent default page reload in form submission.

WHY CONTROLLED FORMS?

- ✓ Easy to validate
- ✓ Track input values
- ✓ Dynamic & predictable
- ✓ Great for complex forms

5 EXAMPLE: CONTROLLED FORM

```
import { useState } from 'react';
function LoginForm() {
  const [form, setForm] = useState({
    email: '',
    password: ''
  });
  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm({ ...form, [name]: value });
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Form Submitted:', form);
    alert('Submitted! Check console 📄');
  };
  return (
    <form onSubmit={handleSubmit} className='form'>
      <div>Login</div>
      <input type='email' name='email' value={form.email}
        onChange={handleChange} placeholder='Email' />
      <input type='password' name='password' value={form.password}
        onChange={handleChange} placeholder='Password' />
      <button type='submit'>Login</button>
    </form>
  );
}
```

JS

How it works:

- 1 User types in input
- 2 onChange updates state
- 3 State value is updated
- 4 UI re-renders with new value
- 5 On submit, handleSubmit is called

UI PREVIEW

Login

Login

6 INPUT TYPES & HANDLING

Input Type	Example	Handling
text	Name	value & onChange
email	user@mail.com	value & onChange
password	*****	value & onChange
checkbox	☒ Accept Terms	checked & onChange
radio	☐ Male ☐ Female	checked & onChange
select	<option>...</option>	value & onChange
textarea	Multi-line text	value & onChange

7 SUBMIT HANDLING

```
const handleSubmit = (e) => {
  e.preventDefault(); // Prevent page reload
  // Do something with form data
  console.log('Form Data:', form);
};
```



Always use `e.preventDefault()` inside `onSubmit`.



9 FREQUENTLY USED EVENTS IN FORMS

- ✓ onChange - When input value changes
- ✓ onFocus - When input gets focus
- ✓ onBlur - When input loses focus
- ✓ onSubmit - When form is submitted
- ✓ onReset - When form is reset



9 BEST PRACTICES

- ✓ Use controlled components.
- ✓ Name your inputs to handle multiple fields easily.
- ✓ Keep form state in a single object.
- ✓ Validate before submit.
- ✓ Show clear error messages.



QUICK TIP

Forms = State + Events

User Input → onChange
→ Update State → Re-render UI



Great forms create great user experiences! Handle events smartly. ❤️

5. CONDITIONAL RENDERING & LISTS

Render the Right UI. Show Data Beautifully. ♥

1 WHAT IS CONDITIONAL RENDERING?

- Allows you to render different UI based on conditions.
- Makes your app dynamic and interactive.
- Used with if statements, operators, and ternary expressions.

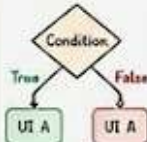


In React, we can control what gets rendered using JavaScript logic.

2 WAYS TO DO CONDITIONAL RENDERING

1. if Statement

Use JS if/else before return.



2. Logical &&

Render if condition is true.



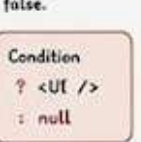
3. Ternary ? :

Choose between two UI elements.



4. Null

Render nothing if condition is false.



Choose the method that makes your code clean and easy to read.

3 CONDITIONAL RENDERING EXAMPLES

A. If Statement

```

if (isLoggedIn) {
  return <Dashboard />;
} else {
  return <Login />;
}
  
```

OUTPUT



B. Logical &&

```

return (
  <div>
    {isLoggedIn && <Welcome back! />}
  </div>
);
  
```

OUTPUT



C. Ternary Operator

```

return (
  <div>
    {isLoggedIn ? (
      <Dashboard />
    ) : (
      <Login />
    )}
  </div>
);
  
```

OUTPUT



4 RENDERING LISTS IN REACT

- Use JavaScript's map() to transform an array into a list of elements.
- Each item in a list must have a unique key.
- Keys help React identify items during updates.

```

const fruits = ['Apple', 'Banana', 'Grape'];
return (
  <div>
    {fruits.map((fruit, index) => (
      <li key={index}>{fruit}</li>
    ))}
  </div>
);
  
```

WHY KEYS?

- Identify items
- Improve performance
- Preserve state
- Avoid bugs

OUTPUT

- Apple
- Banana
- Grape



Always add a unique key to each item in the list.

5 LIST RENDERING EXAMPLES

```

const users = [
  { id: 1, name: 'Mitli' },
  { id: 2, name: 'Rohan' },
  { id: 3, name: 'Sneha' },
];
return (
  <div>
    {users.map(user => (
      <li key={user.id}>
        {user.name}
      </li>
    ))}
  </div>
);
  
```

OUTPUT

- Mitli
- Rohan
- Sneha

TIPS

- Use unique id for keys.
- Avoid using index as key if items can change order.
- Keys should be stable & unique.



6 CONDITIONAL RENDERING WITH LISTS

Show List if Not Empty

```

const items = [1, 2, 3];
return (
  <div>
    {items.length > 0 ? (
      <div>
        {items.map(item => (
          <li key={item}>{item}</li>
        ))}
      </div>
    ) : (
      <p>No items found.</p>
    )}
  </div>
);
  
```

OUTPUT (items.length > 0)

- 1
- 2
- 3

OUTPUT (items empty)

No items found.



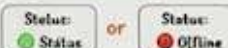
Use ternary operator to show fallback UI.

7 TERNARY OPERATOR IN JSX

```

const isOnline = true;
return (
  <div>
    {<div>
      {isOnline ? 'Online' : 'Offline'}
    </div>}
  </div>
);
  
```

OUTPUT



Ternary is perfect for small conditional UI.

8 DON'T DO THIS!

Avoid using index as key when list order can change.

```

{items.map((item, index) => (
  <li key={index}>{item}</li>
))}
  
```



Do this:

```

{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
  
```



9 QUICK SUMMARY

Conditional Rendering

if
Logical &&
Ternary ? :
null

Lists

Use map()
Always use unique key
Keys improve performance
Avoid index as key

6. useEffect & SIDE EFFECTS

Synchronize with External World. Fetch. Listen. Clean up. ❤️

1 WHAT IS useEffect?

- useEffect is a React Hook that lets you perform side effects in function components.
- Runs after the component renders.
- Used to fetch data, subscribe to events, manipulate DOM, timers, and more.



If rendering is the "what", useEffect is the "when" you need to do something after render.

2 SYNTAX

```
useEffect(() => {
  // Side effect code here
  return () => {
    // Cleanup code (optional)
  };
}, [dependencies]); // Optional dependencies array
```

- Runs after render (not during render)
- Runs on mount, update (based on dependencies), and unmount (cleanup)

PARAMETERS

Effect: Function to run after render
Cleanup: Function to run before next effect or on unmount
Dependencies: Array of values to watch



3 WHEN DOES useEffect RUN?

Dependency Array	When it Runs	Example Use
No array useEffect(() => { ... });	After every render (mount + every update)	Rarely used (may cause loops)
Empty array [] useEffect(() => { ... }, []);	Only once (after first render)	Fetch data on mount, set up subscriptions
[dep1, dep2] useEffect(() => { ... }, [dep1, dep2]);	When any dependency changes	Re-fetch data when dep changes



Always include only necessary dependencies!

4 EXAMPLES

A. Run Once (on Mount)

```
useEffect(() => {
  console.log("Component mounted!");
}, []);
```

Great for:
API calls, setup, subscriptions

B. Run on Dependency Change

```
useEffect(() => {
  console.log("Search changed!");
  search();
  fetchData(search);
}, [search]);
```

Runs when "search" changes

C. After Every Render

```
useEffect(() => {
  document.title = `Counter: ${count}`;
});
```

Updates document title on every render



Keep effects focused and avoid unnecessary work.



5 CLEANUP FUNCTION

- Cleanup runs before the effect runs again or when the component unmounts.
- Used to remove event listeners, clear timers, cancel requests, etc.



Cleanup +
Avoid memory leaks!

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("tick");
  }, 1000);
  // cleanup
  return () => clearInterval(timer);
}, []);
```

LIFECYCLE FLOW



6 COMMON USE CASES

Fetch Data	Event Listeners	Timers	Subscriptions	DOM Updates
 Get data from APIs	 Add/remove listeners	 setTimeout, setInterval	 WebSocket, Firebase, etc.	 Manually update document



If it affects something outside of rendering, it probably belongs in useEffect.



7 BEST PRACTICES

- Always add the right dependencies.
- Don't mutate dependencies inside the effect.
- Use cleanup to prevent memory leaks.
- Split effects: One effect = One responsibility.
- Use ESLint plugin (react-hooks) to avoid mistakes.

GOOD ✓

```
useEffect(() => {
  fetchData(userId);
}, [userId]);
```

BAD ✗

```
useEffect(() => {
  fetchData(userId);
}, []); // State userId!
```

8 QUICK REFERENCE

Pattern	Code	Runs
No Dependency	useEffect(() => { ... });	After every render
Empty Array	useEffect(() => { ... }, []);	Once on mount
With Dependencies	useEffect(() => { ... }, [a, b]);	When a or b changes
Cleanup	useEffect(() => { return () => { ... }; }, [dep]);	Before next effect or on unmount



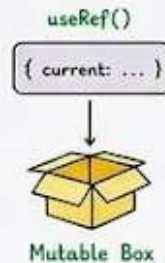
useEffect is powerful, but with great power comes great responsibility! ❤️

7. useRef & DOM REFERENCES

Access DOM, Persist Values. Focus. Measure. Much More! ❤️

1 WHAT IS useRef?

- useRef is a React hook that returns a mutable ref object.
- The ref object has a `.current` property.
- Changing `.current` does NOT trigger re-render.
- Perfect for accessing DOM, storing values, timers, previous values, etc.



Think of useRef as a box whose content you can change anytime without re-rendering.

2 SYNTAX

```

import { useRef } from 'react';

function MyComponent() {
  const myRef = useRef(initialValue);

  return (
    <div>...</div>
  );
}
    
```

PARAMETERS

initialValue: The initial value for the ref. If not provided, it's set to null.



The returned object stays the same between renders.

3 ACCESSING DOM ELEMENTS

```

import { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} type="text"
        placeholder="I will be focused" />
      <button onClick={focusInput} type="button">
        Focus Input
      </button>
    </div>
  );
}
    
```

How it works:

- Create a ref with `useRef(null)`
- Attach it to an element using `ref`
- Access the element using `ref.current`
- Call any DOM method on `ref.current`

OUTPUT

I will be focused

Focus Input

4 PERSISTING VALUES (WITHOUT RE-RENDER)

```

function Counter() {
  const renderCount = useRef(0);

  const handleClick = () => {
    renderCount.current = renderCount.current + 1;
    console.log('Button clicked!', renderCount.current);
  };

  return (
    <button onClick={handleClick}>Click Me</button>
  );
}
    
```

EXPLANATION

- ✓ `renderCount.current` updates but does NOT cause re-render.
- ✓ Value persists across renders.



Great for storing values like previous state, timers, IDs, or anything that doesn't need to trigger UI updates.

5 COMMON USE CASES

Focus	Store Values	Previous Values	Timers	DOM Measure size
Focus input, buttons, etc.	Store mutable values without re-render.	Store previous state or props.	Store timer IDs for <code>setInterval</code> / <code>setInterval</code> .	Measure size, position, etc.

6 MEASURING DOM ELEMENTS

```

import { useRef, useEffect } from 'react';

function MeasureBox() {
  const boxRef = useRef(null);
  const [size, setSize] = useState({});

  useEffect(() => {
    if (boxRef.current) {
      const { width, height } = boxRef.current.getBoundingClientRect();
      setSize({ width, height });
    }
  }, []);

  return (
    <div ref={boxRef} style={{ padding: '5px', background: 'lightblue', border: '1px solid #3498db' }}>
      Measure Me
    </div>
  );
}
    
```

OUTPUT (Example)

Measure Me

Size:
Width: 300px
Height: 100px



`getBoundingClientRect()` gives size and position information.

7 FORWARDING REFS

- By default, refs do not pass to child components.
- Use `forwardRef` to forward refs to DOM nodes.

```

// Parent Component
const InputRef = useRef(null);

return <CustomInput ref={InputRef} />;

// Child Component
const CustomInput = forwardRef((props, ref) => {
  return <input ref={ref} {...props} />;
});

// Usage
forwardRef((props, ref) => ...) lets parent access child's DOM.
    
```



8 BEST PRACTICES

- ✓ Use `useRef()` for mutable values that don't affect UI.
- ✓ Do not measure refs.
- ✓ Prefer state for data that affects UI.
- ✓ Always check if `ref.current` exists.

DO



```

const inputRef = useRef(null);
useEffect(() => {
  inputRef.current.focus();
}, []);
    
```

DON'T



```

if (inputRef) {
  inputRef.focus();
} // Wrong! should use inputRef.current
    
```



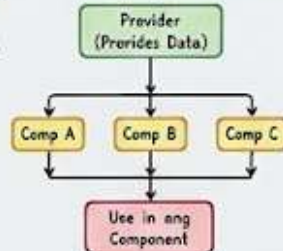
useRef is your pocket tool for DOM access and storing values without re-rendering. ❤️

8. useContext & CONTEXT API

Share Data. Avoid Prop Drilling. Keep It Clean! ❤️

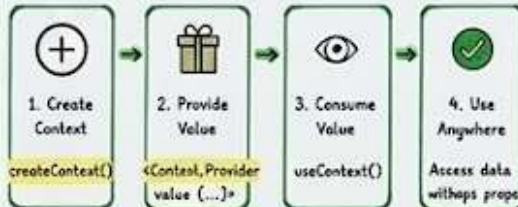
1 WHAT IS CONTEXT API?

- Context Hook is a way to pass data through the component tree without passing props down manually at every level.
- It's perfect for global data like theme, user, language, auth, etc.



Context is not a state management text.
It's for pasting data, not managing complex state.

2 HOW DOES IT WORK?



When the value in **Provider** changes, all consuming components re-render.

WHEN TO USE?

- ✓ Theme (light/dark)
- ✓ User Authentication
- ✓ Language Settings
- ✓ App Configurations
- ✓ App global data

3 STEP 1: CREATE CONTEXT

```
// Create Context
import { createContext } from 'react';
const MyContext = createContext();
```

```
// Default value (optional)
const MgContext = createContext({
  user: null,
  theme: 'light'
});
```

NOTE

The default value is used only when a component is used outside of any Provider.

4 STEP 2: PROVIDE VALUE

```
import { createContext, useState } from 'react';
export const MyContext = createContext();
export function MyProvider({ children }) {
  const [user, setUser] = useState({ name: 'Mitoli' });
  const [theme, setTheme] = useState('Mitoli');
  const value = { user, setUser, theme, setTheme };
  return (
    <MyContext.Provider value={value}>
      {children}
    </MyContext.Provider>
  );
}
```

KEY POINTS

- ✓ Wrap your app for part of it) with Provider.
- ✓ The value prop can be any data (object, string, number, functions, etc.).

5 STEP 5: CONSUME WITH `useContext`

```
import { useContext } from 'react';
import { MyContent } from './MyProvider';
```

```
function Profile() {
  const { user, theme, setTheme } = useContext(MyContext);

  return (
    <div style={{ value: theme === 'light' ? '8000' : '0fff' }}>
      <div user={user.name}>
        <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
          Toggle Theme
        </button>
      </div>
    </div>
  );
}
```

NOTE

useContext must be used inside a component that is wrapped by the Provider.

6 COMPLETE EXAMPLE



```
// App.js
import { MyProvider } from './DevWithRishabh';
import Profile from './components/Profile';

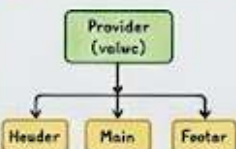
function App() {
  return (
    <MyProvider>
      <React Context API />
      <Profile />
    </MyProvider>
  );
}

export default App;
```



Now Profile (and any child) can assess user & theme without receiving props at every level!

7 MULTIPLE CONSUMEES



- ✓ Any component inside Provider can consume the context.
- ✓ Helps in deeply nested component tree.

8 BEST PRACTICES

- ✓ Keep context value nominal & focused.
- ✓ Memoize value object to avoid unnecessary re-renders.
- ✓ Don't use context for frequent changes (like input typing).
- ✓ Split context if data is not related.
- ✓ Combine with useCallback for complex logic.

```
// Memoise value
const value = useMemo(() => {
  { user, setlicher, theme, setTheme }
}, [user, theme]);

return (
  <MyContext.Provider value={value}>
    {children}
  </MyContext.Provider>
);
```

9 QUICK REFERENCE

```
Create Context
const MyContext =
  createContext();
```

```

    Provide Value
    <MyContext.Provider
    value={value}>
      (children)
    </MyContext.Provider>

```

```
consume
const value =
useContext(MyContext);
```

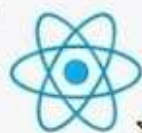
Default Value
<code>createContext(defaultValue)</code>

Update Value
Change state inside Provider
→ Consumers re-render

Don't Forget
Woop comparers with
the Provider!

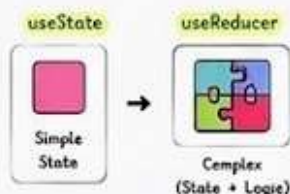
9. useReducer & COMPLEX STATE

Manage Complexity. Keep Logic Predictable. Scale with Confidence!



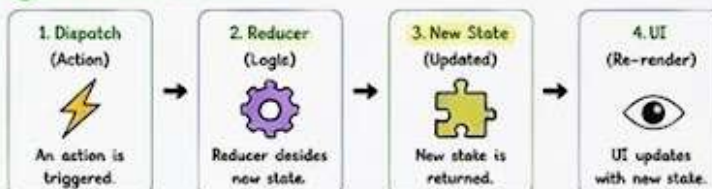
1 WHY useReducer?

- useReducer is a React Hook that manages complex state logic.
- Perfect for multiple sub-values or related state updates.
- State updates are predictable and easier to test.
- Inspired by Redux, built into React.



If your state is simple → useState
If your state is complex → useReducer

2 HOW DOES IT WORK?



Action → Reducer → New State → UI Update

3 SYNTAX

```

import { useReducer } from 'react';
const initialState = ( /* initial state */ );
function reducer(state, action) {
  // return new state based on action
}
const [state, dispatch] = useReducer(reducer, initialState);
  
```



state → current state
dispatch → function used to send actions

4 ANATOMY OF REDUCER

```

function reducer(state, action) {
  switch (action.type) {
    case 'TYPE_1':
      return { ...state, /* updates */ };
    case 'TYPE_2':
      return { ...state, /* updates */ };
    // more cases...
    default:
      return state; // Always return a state
  }
}
  
```

KEY POINTS

- ✓ Takes current state & action.
- ✓ Returns new state (must not mutate).
- ✓ Action is an object with a type.
- ✓ Default case returns current state.



5 COMPLETE EXAMPLE: COUNTER

```

import { useReducer } from 'react';
const initialState = { count: 0 };
function reducer(state, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    case 'DECREMENT':
      return { count: state.count - 1 };
    case 'RESET':
      return { count: 0 };
    default:
      return state;
  }
}
function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);
  return (
    <div className="border">
      <div style="border: 1px solid #ccc; padding: 5px; text-align: center;>
        <button onClick={() => dispatch({ type: 'INCREMENT' })}> +1 </button>
        <button onClick={() => dispatch({ type: 'DECREMENT' })}> -1 </button>
        <button onClick={() => dispatch({ type: 'RESET' })}> Reset </button>
      </div>
      <div style="margin-top: 10px;>
        Count: {state.count}
      </div>
    </div>
  );
}
export default Counter;
  
```

How it works:

- 1 User clicks a button
- 2 An action is dispatched.
- 3 Reducer returns a new state.
- 4 Component re-renders with updated state.

UI PREVIEW

5



6 TODO APP EXAMPLE (MORE COMPLEX)

```

const initialState = {
  todos: [],
  filter: 'all'
};
function reducer(state, action) {
  switch (action.type) {
    case 'ADD_TODO':
      return {
        todos: [...state.todos, { id: Date.now(), text: action.text, done: false }]
      };
    case 'TOGGLE_TODO':
      return {
        ...state,
        todos: state.todos.map(todo =>
          todo.id === action.id ? { ...todo, done: !todo.done } : todo
        )
      };
    case 'SET_FILTER':
      return { ...state, filter: action.filter };
    default:
      return state;
  }
}
  
```



Great for apps with multiple values (todos, filters, etc.) and complex update logic.



7 DISPATCHING ACTIONS

```

dispatch({ type: 'ADD_TODO', text: 'Learn React' });
dispatch({ type: 'TOGGLE_TODO', id: 1 });
dispatch({ type: 'SET_FILTER', filter: 'completed' });
  
```



Actions are plain objects.
You can add extra data (payload).



8 WHEN TO USE WHICH?

Use useState when...

- ✓ State is simple
- ✓ Few values
- ✓ Updates are straightforward
- ✓ Example: input value, toggle

Use useReducer when...

- ✓ State logic is complex
- ✓ Multiple sub-values
- ✓ Many update types
- ✓ Example: forms, lists, wizards



9 BEST PRACTICES

- ✓ Keep reducer pure (no side effects).
- ✓ Always return new state (don't mutate).
- ✓ Use meaningful action types.
- ✓ Group related state logic in one reducer.
- ✓ Combine with Context API for global state.



10 QUICK REFERENCE

Item	Example	Description
Hook	<code>const [state, dispatch] = useReducer(reducer, initialState);</code>	Introduces reducer with state and dispatch.
Reducer Args	<code>(state, action)</code>	state = current state, action = dispatched action
Action Shape	<code>{ type: 'ACTION_TYPE', payload: any }</code>	type is required, payload is optional
Return	New State	Reducer must return the new state
Default Case	<code>default: return state;</code>	Always return state for unknown actions

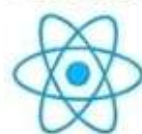


BONUS TIP

Think of useReducer like a mini Redux inside your component.
It brings structure, clarity, and scalability to your state management!



10. Custom Hooks



Extract Logic. Reuse Anywhere. Keep Components Clean! ❤️

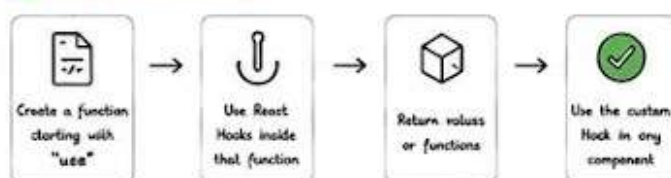
1 WHAT ARE CUSTOM HOOKS?

- Custom Hooks are JavaScript functions whose name starts with "use".
- They let you extract and reuse stateful logic between components.
- They can use other Hooks inside.
- They help keep components clean and focused.



Think of Custom Hooks as your own built-in Hooks!

2 HOW DO THEY WORK?



Custom Hooks follow the same Rules of Hooks!

3 SYNTAX

```

function useSomething(params) {
  // use state, useEffect, useRef, etc.
  const [value, setValue] = useState(initialValue);

  // your custom logic here

  return { value, setValue };
}
  
```



Most start with "use" and can only be called inside React function components or other Hooks.

4 EXAMPLE 1: useWindowSize

```

import { useState, useEffect } from 'react';

function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    function handleResize() {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    }
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return size;
}

export default useWindowSize;
  
```

Usage

```

import useWindowSize from './useWindowSize';

function App() {
  const { width, height } = useWindowSize();

  return (
    <div>
      <p>Width: {width}</p>
      <p>Height: {height}</p>
    </div>
  );
}
  
```



Reuses window size logic anywhere!



5 EXAMPLE 2: useFetch

```

import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    let isMounted = true;
    async function fetchData() {
      try {
        const res = await fetch(url);
        const json = await res.json();
        if (!isMounted) return;
        setData(json);
      } catch (err) {
        if (isMounted) setError(err);
      } finally {
        if (isMounted) setLoading(false);
      }
    }
    fetchData();
    return () => { isMounted = false; };
  }, [url]);

  return { data, loading, error };
}

export default useFetch;
  
```

Usage

```

import useFetch from './useFetch';

function User() {
  const { data, loading, error } = useFetch('https://jsonplaceholder.typicode.com/users/1');

  if (loading) return <Loading... />;
  if (error) return <Error: {error.message} />;

  return (
    <div>
      {data.map(user => {
        <div>{user.id}</div> {user.name}</div>
      })}
    </div>
  );
}
  
```



Reusable data fetching with loading & error handling!



6 EXAMPLE 3: useToggle

```

import { useState, useCallback } from 'react';

function useToggle(initial = false) {
  const [value, setValue] = useState(initial);

  const toggle = useCallback(() => setValue(v => !v), []);

  const setTrue = useCallback(() => setValue(true), []);
  const setFalse = useCallback(() => setValue(false), []);

  return [value, toggle, setTrue, setFalse];
}

export default useToggle;
  
```

Usage

```

import useToggle from './useToggle';

function ThemeSwitcher() {
  const [isDark, toggle, setDark, setLight] = useToggle(false);

  return (
    <div>
      <p>Theme: {isDark ? 'Dark' : 'Light'}</p>
      <button onClick={toggle}>Toggle Theme</button>
      <button onClick={setDark}>Dark</button>
      <button onClick={setLight}>Light</button>
    </div>
  );
}
  
```

Great for:

- ✓ Reusable UI logic
- ✓ Cleaner components
- ✓ Better separation of concerns
- ✓ Easier testing
- ✓ Consistent behavior



7 RULES FOR CUSTOM HOOKS

- Start with "use".
- Only call Hooks at the top level.
- Only call from React function components or other Hooks.
- Don't call Hooks inside loops, conditions, or nested functions.

Remember!

Custom Hooks make your code DRY, reusable and powerful! ❤️

8 BEST PRACTICES

- Keep Hooks focused on one job.
- Return only what the component needs.
- Use meaningful names (useUser, useForm, etc.).
- Keep dependencies correct in useEffect.
- Add JSDoc comments for better clarity.



9 WHEN TO USE?

- ✓ When logic is reused in multiple places.
- ✓ When component becomes too large.
- ✓ When stateful logic needs to be shared.
- ✓ When you want cleaner, testable code.



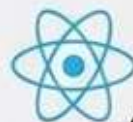
BONUS TIP: Build your own hook library over time. Your future self will thank you! 🚀



</> DevWithRishabh </>

</> DevWithRishabh </>

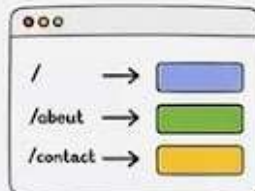
11. React Router & Navigation



Build SPA Navigation, Smooth Routes, Better UX! ❤️

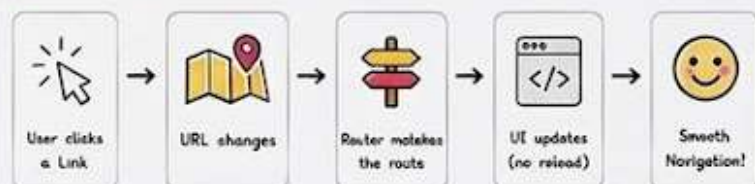
1 WHAT IS REACT ROUTER?

- React Router is the standard routing library for React apps.
- It enables navigation between different views (routes) without refreshing the page.
- Helps build Single Page Applications (SPAs).



Navigate without reload. Keep your app fast!

2 HOW DOES IT WORK?



URL → Route Match → Component Render

3 INSTALLATION

Using npm
npm install react-router-dom

Using yarn
yarn add react-router-dom

We'll use
react-router-dom
in all examples.

4 SETUP: BROWSER ROUTER

```

import { BrowserRouter } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      /* All your routes here */
    </BrowserRouter>
  );
}
  
```

KEY POINTS

- ✓ BrowserRouter uses HTML5 history API.
- ✓ Use it at the top level of your app.
- ✓ Wrap all <Routes> inside it.

5 ROUTES & ROUTE PATHS

```

import { Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={Home} />
        <Route path="/about" element={About} />
        <Route path="/contact" element={Contact} />
      </Routes>
    </BrowserRouter>
  );
}
  
```

Notes

- ✓ Routes is a container for all Route.
- ✓ element prop renders the component.
- ✓ "/" is the home route.

6 NAVIGATION: <Link> & useNavigate()

Using <Link> (Declarative)

```

import { Link } from 'react-router-dom';

function Navber() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
      <Link to="/contact">Contact</Link>
    </nav>
  );
}
  
```

Using useNavigate() (Programmatic)

```

import { useNavigate } from 'react-router-dom';

function GoToAbout() {
  const navigate = useNavigate();
  const handleClick = () => {
    navigate('/about');
  };

  return (
    <button onClick={handleClick}>
      Go to About
    </button>
  );
}
  
```



Use <Link> for links. Use useNavigate() for actions (buttons, form submit, etc.).

7 ACTIVE LINKS

```

import { NavLink } from 'react-router-dom';

const
  <NavLink to="/" className={() => {active ? 'active' : ''}} >
    Home
  </NavLink>
</nav>
  
```

TIP

NavLink adds an "active" class when the link matches the current URL.

8 NESTED ROUTES

```

import { Routes, Route, Outlet } from 'react-router-dom';

function Dashboard() {
  return (
    <div>
      <h2>Dashboard</h2>
      <div>
        <Link to="/dashboard/overview">Overview</Link>
        <Link to="/dashboard/stats">Stats</Link>
      </div>
      <div>
        <Outlet />
      </div>
    </div>
  );
}
  
```

Route Structure

```

/dashboard
├── overview
└── stats

<Route path="/dashboard"
  element={Dashboard} />
  <Route path="/overview"
    element={Overview} />
  <Route path="/stats"
    element={Stats} />
</Route>
  
```

9 404 - NOT FOUND PAGE

```

<Routes>
  /* other routes */
  <Route path="*" element={NotFound} />
</Routes>
  
```

Use * to catch unmatched paths and show a 404 page.

404

10 BEST PRACTICES

- ✓ Keep routes clean and organized.
- ✓ Use meaningful paths (no random ids).
- ✓ Use NavLink for active navigation.
- ✓ Handle 404 routes for better UX.
- ✓ Lazy load pages for large apps.
- ✓ Keep navigation logic in one place.



BONUS TIP: React Router makes your app feel native. Master it for professional SPA development! ❤️

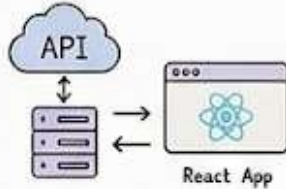
12. API Calls & Data Fetching



Get Data. Handle State. Show Results.

1 WHAT & WHY?

- ✓ APIs let your app talk to servers and get or send data.
- ✓ Common over show lists, details, users, products, etc.
- ✓ In React, we fetch data and store it in state to display.



Think: API = Data Source ↔ React = UI

2 THE BASIC FLOW



Fetch → Response → State → UI

3 EXAMPLE: FETCH WITH useEffect()

```

import { useState, useEffect } from 'react';

function Posts() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function fetchData() {
      try {
        const res = await fetch('https://jsonplaceholder.typicode.com/posts');
        if (!res.ok) throw new Error('Failed to fetch data');
        const data = await res.json();
        setPosts(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    fetchData();
  }, []); // runs once on mount

  if (loading) return <div>loading...</div>;
  if (error) return <div>error: {error}</div>;

  return (
    <div>
      {posts.map(post => (
        <div key={post.id}>
          {post.title} - {post.body}
        </div>
      ))}
    </div>
  );
}

export default Posts;
  
```

WHAT'S HAPPENING?

- 1 Component mounts.
- 2 useEffect runs.
- 3 fetch() makes API call.
- 4 Response converted to JSON.
- 5 State updated with data.
- 6 Component re-renders with the data.



4 USING AXIOS (ALTERNATIVE)

```

// install: npm install axios
import { useState, useEffect } from 'react';
import axios from 'axios';

function Posts() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    async function getPosts() {
      try {
        const res = await axios.get('https://jsonplaceholder.typicode.com/posts');
        const data = res.data;
      } catch (err) {
        console.error(err);
      }
    }
    getPosts();
  }, []);

  return (
    <div>
      {posts.map(post => (
        <div key={post.id}>
          {post.title} - {post.body}
        </div>
      ))}
    </div>
  );
}

export default Posts;
  
```

WHY AXIOS?

- ✓ Cleaner syntax
- ✓ Automatic JSON parsing
- ✓ Better error handling
- ✓ Request/Response interceptors

axios

5 HANDLING STATES

State	When	Example UI
Loading	API call in progress	loading...
Success	Data fetched successfully	Show data in UI
Error	Something went wrong	error: {error}

6 PAGINATION EXAMPLE

```

const [page, setPage] = useState(1);

useEffect(() => {
  fetch('https://jsonplaceholder.typicode.com/posts?_page=${page}&_limit=10')
    .then(res => res.json())
    .then(data => setPosts(data));
}, [page]);

return (
  <div>
    <button onClick={() => setPage(p => p - 1)} disabled={page === 1}>Previous</button>
    <span>Page {page} of {count}</span>
    <button onClick={() => setPage(p => p + 1)} disabled={page === count}>Next</button>
  </div>
);
  
```

Notes

- ✓ APIs often support pagination.
- ✓ Change page → new data.
- ✓ Great for long lists.

< 1 2 3 >

7 BEST PRACTICES

- ✓ Always handle loading & error states.
- ✓ Don't forget to handle API errors.
- ✓ Use meaningful variable names.
- ✓ Cancel requests if component unmounts (optional).
- ✓ Keep API keys & secrets safe.



8 USEFUL SNIPPETS

```

• Base URL
const BASE_URL = 'https://api.example.com';

• Headers (Custom)
const options = {
  headers: { 'Content-Type': 'application/json' }
};

• POST Request (Fetch)
await fetch(BASE_URL, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ /* data */ })
});
  
```

9 COMMON STATUS CODES

Code	Meaning	Use
200	OK	Success
201	Created	Resource created
400	Bad Request	Invalid request
403	Unauthorized	Auth required
404	Not Found	Resource not found
500	Server Error	Something broke



BONUS TIP: Create a custom hook like useFetch() to reuse your logic across components!



</> DevWithRishabh </>

13. Performance Optimization — memo, useMemo & useCallback

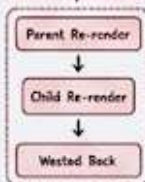


Faster Apps. Better Experience. Smarter Rendering ❤️

1 WHY OPTIMIZE?

- React re-renders components when state or props change.
- Unnecessary re-renders can slow down your app.
- Use memo, useMemo and useCallback to boost performance.

Without Optimization



With Optimization



Optimize only when needed, not everywhere!

2 OVERVIEW

Hook / HOC	Propose	When to Use	Key Benefit
React.memo	Memoizes a component	When component re-renders with same props	Prevents unnecessary re-renders
useMemo	Memoizes a computed value	When expensive calculation depends on some values	Avoids expensive re-calculations
useCallback	Memoizes a function reference	When passing callbacks to child components	Prevents child re-renders due to new function refs



All three help in reducing unnecessary work.

3 React.memo (MEMOIZE COMPONENTS)

Prevents functional components from re-rendering if props haven't changed.

```
// ChildComponent.js
const ChildComponent = React.memo((props) => {
  console.log("Child rendered");
  return (
    <div>
      <div>{props.name}</div>
      <p>Age: {props.age}</p>
    </div>
  );
});
```

How it works:

React memo performs a shallow comparison of props. If props are same → component doesn't re-render.

4 useMemo (MEMOIZE VALUES)

Memoizes expensive calculations and returns the cached result until dependencies change.

```
import { useMemo } from "react";
function ExpensiveComponent({ items }) {
  // Expensive calculation
  const total = useMemo(() => {
    console.log("Calculating total...");
    return items.reduce((total, item) => {
      sum + item.price, 0);
    }, [items]); // Recalculate only when items change
  }, [items]);
  return <div>Total: $<total></div>;
}
```

Use when the calculation is costly and depends on specific values.



5 useCallback (MEMOIZE FUNCTIONS)

Memoizes a function to its reference stops the same with dependencies change.

```
import { useCallback } from "react";
function Parent({ onAction, items }) {
  const handleClick = useCallback((id) => {
    console.log("Item clicked.", id);
    onAction(id);
  }, [onAction]); // function ref changes only // when onAction changes
  return <ChildComponent onClick={handleClick} items={items} />;
}
```

Use when passing functions to child components that rely on reference equality (e.g., memo).



6 PRACTICAL EXAMPLE (PUTTING IT TOGETHER)

```
function Parent() {
  const [count, setCount] = useState(0);
  const [items, setItems] = useState([
    { id: 1, name: "Book", price: 200 },
    { id: 2, name: "Pen", price: 50 }
  ]);

  // 1. useMemo: calculate total only when items change
  const total = useMemo(() => {
    const reduce((sum, item) => sum + item.price, 0);
  }, [items]);

  // 2. useCallback: memoize function passed to child
  const handleClick = useCallback((id) => {
    console.log("Clicked item", id);
  }, []);

  return (
    <div>
      <div>Count: {count}</div>
      <p>Total: {total}</p>
      <button onClick={() => setCount(c => c + 1)}>
        Increment Count
      </button>
      <ChildComponent items={items} onClick={handleClick} />
    </div>
  );
}
```

```
Itemlist Component
const ItemList = React.memo((props) => {
  const { items, handleClick } = props;
  console.log("ItemList rendered");
  return (
    <div>
      {items.map(item => (
        <div>
          {item.name} - {item.price}
          <button onClick={() => handleClick(item.id)}>
            Click
          </button>
        </div>
      ))}
    </div>
  );
});
```

New ItemList re-renders only when items or handleClick actually change!

7 WHEN TO USE?

- Use React.memo for pure presentational components.
- Use useMemo for expensive calculations (filtering, sorting, complex logic).
- Use useCallback when passing functions to memoized child components.



Don't over-optimize!

- Premature optimization can make code complex.
- Measure performance before optimizing.



8 SHALLOW COMPARISON (IMPORTANT!)

React.memo uses shallow comparison:

- Compares prop references, not deep values.
- Works great with primitive values.
- For objects/arrays, keep references stable.

Example

```
<div data={ { id: 1 } } />
(New reference → re-render) ❌

useMemo(() => { { id: 1 }, {} })
(Stable reference → skip render) ✅
```

9 BEST PRACTICES

- Profile first, optimize second.
- Keep components pure.
- Don't pass new objects/arrays/functions unless necessary.
- Use dependency arrays correctly.



10 QUICK REFERENCE

Hook / HOC	Syntax	Example Use Case
React.memo	React.memo(Component)	Memoize child component
useMemo	useMemo(() => value, [deps])	Expensive calculations
useCallback	useCallback(fn, [deps])	Callbacks passed to children

11 KEY TAKEAWAY

memo, useMemo, and useCallback help you write high-performance React apps by reducing unnecessary re-renders and computations. Use them smartly for maximum impact!



BONUS TIP: Use React DevTools Profiler to find performance bottlenecks in your app! ❤️



14. Advanced React – Composition, Error Boundaries & Patterns

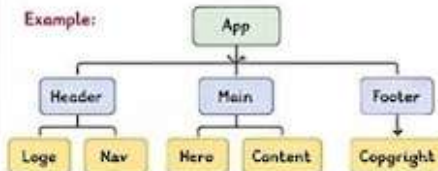
Build Scalable, Maintainable, Resilient React Apps!



1 COMPONENT COMPOSITION

Break UI into small, reusable pieces and compose them to build complex UIs.

Example:



Benefits

- ✓ Reusability
- ✓ Readability
- ✓ Maintainability
- ✓ Testability



Think in building blocks. Compose, don't duplicate.

2 CHILDREN & PROPS COMPOSITION

Using children

```

<Card>
  <h3>Title</h3>
  <p>Some content...</p>
</Card>
  
```

Using props

```

<Button
  text="Click me"
  variant="primary"
  onClick={handleClick}
/>
  
```



Use children for flexible content.
Use props for configurable behavior.

3 ERROR BOUNDARIES

Catch JavaScript errors anywhere in the child component tree, log those errors, and display a fallback UI.

```

import React from 'react';

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  componentDidCatch(error, info) {
    console.error("Error caught:", error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h3>Something went wrong. Please try again.</h3>;
    }
    return this.props.children;
  }
}

export default ErrorBoundary;
  
```

Usage

```

<ErrorBoundary>
  <App />
</ErrorBoundary>
  
```

Notes

- ✓ Only class components can be error boundaries.
- ✓ It catches errors in child component tree.
- ✓ Does not catch errors in event handlers or SSR.



4 HIGHER ORDER COMPONENTS (HOC)

A function that takes a component and returns a new component with added props or behavior.

```

function withLoading(WrappedComponent) {
  return function WithLoadingComponent(props) {
    const [loading, setLoading] = React.useState(true);

    React.useEffect(() => {
      setTimeout(() => setLoading(false), 1000);
    }, []);

    if (loading) return <div>Loading...</div>;
    return <WrappedComponent {...props} />;
  };
}

export default withLoading;
  
```

Usage

```

const UserList =
  withLoading(
    UserListBase
  );
  
```

Best for

- ✓ Reusing logic
- ✓ Code reuse
- ✓ Cross-cutting concerns

5 RENDER PROPS PATTERN

```

function Player({ render }) {
  const [position, setPosition] = React.useState({ x: 0, y: 0 });

  React.useEffect(() => {
    const handleMouseMove = (e) => {
      setPosition({ x: e.clientX, y: e.clientY });
    };
    window.addEventListener("mousemove", handleMouseMove);
    return () => window.removeEventListener("mousemove", handleMouseMove);
  }, []);

  return render(position);
}

// Usage
<Player render={((x, y) => <div>Player at: {x}, {y}</div>)} />
  
```

Best for

- ✓ Sharing stateful logic
- ✓ Reusability
- ✓ Highly Reusable



6 CUSTOM HOOKS (FOR COMPOSITION)

```

function useToggle(initial = false) {
  const [value, setValue] = React.useState(initial);
  const toggle = React.useCallback(() => setValue(v => !v), []);
  return [value, toggle];
}

// Usage
function App() {
  const [isOn, toggle] = useToggle();
  return (
    <button onClick={toggle}>{isOn ? 'OFF' : 'ON'}</button>
  );
}
  
```

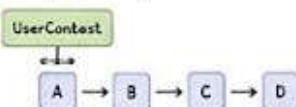
Why?

- ✓ Encapsulate logic
- ✓ Reusable across components
- ✓ Cleaner code & separation of concerns



7 CONTEXT FOR COMPOSITION

Share data across deep component trees without prop drilling.



Use Context + Custom Hooks for powerful and clean state sharing.



8 OTHER IMPORTANT PATTERNS

Pattern	Use Case
Compound Components	Build flexible & explicit APIs (e.g., <Tabs>{Tabs.List}...)
Controlled Components	Form handling, inputs, state management
Function on Child (Render Props)	Share dynamic values via function
Custom Hooks	Extract & reuse stateful logic
Error Boundaries	Handle runtime errors gracefully

9 BEST PRACTICES

- ✓ Prefer composition over inheritance.
- ✓ Keep components small and focused.
- ✓ Use Error Boundaries to prevent crashes.
- ✓ Extract repeating logic into Custom Hooks.
- ✓ Use Context sparingly and wisely.
- ✓ Memoize only when needed.
- ✓ Follow single responsibility principle.



Good structure today, scalable app tomorrow!



BONUS TIP: Composition + Reusability + Resilience = Production-ready React Apps!



15. React Master Reference



Everything in One Place. Build with Confidence! ❤️

1 CORE CONCEPTS

- Components: Reusable UI Blocks.
- JSX: Write HTML inside JS.
- Props: Data passed from parent to child.
- State: Data managed inside a component.
- One-way Data Flow.
- Virtual DOM: Fast & efficient updates.



2 ESSENTIAL HOOKS

Hook	Purpose
useState	Add state to functional components
useEffect	Handle side effects
useReducer	Share data across the component tree
useMemo	Manage complex state logic
useCallback	Memoise expensive calculations
useRef	Memoise functions
useCallback	Memoise functions
useRef	Persist values between renders
	Access DOM elements

3 COMPONENT PATTERNS

- Presentational / Dumb UI only, gets data via props.
- Container / Smart Handles data & logic.
- Compound Components Use children/props to build flexible APIs.
- Render Props Share logic using a function prop.
- Higher Order Components (HOC) Enhance components with extra props.
- Custom Hooks Extract & reuse logic across components.

4 ROUTING (react-router-dom)

Import { BrowserRouter, Router, Route, Link } from 'react-router-dom';

```

function App() {
  return (
    <BrowserRouter>
      <Link to="/">Home</Link>
      <Link to="/About">About</Link>
    </BrowserRouter>
  );
}
  
```

Key Points

- ✓ Use <Link> for navigation.
- ✓ Use <Route> and <Router>.
- ✓ Supports nested routes.
- ✓ Great for SPAs.



5 DATA FETCHING

With useEffect + fetch

```

const [data, setData] = useState();

useEffect(() => {
  fetch('https://api.example.com/data')
    .then(res => res.json())
    .then(setData)
    .catch(err => console.error(err));
}, []);
  
```

Best Practices

- ✓ Handle loading & error states.
- ✓ Cancel requests on unmount.
- ✓ Use meaningful variable names.
- ✓ Consider Libraries: React Query, SWR.



With axios

```

import axios from 'axios';

useEffect(() => {
  axios.get('https://api.example.com/data')
    .then(res => setData(res.data))
    .catch(err => console.error(err));
}, []);
  
```

Why axios?

- ✓ Cleaner syntax
- ✓ Automatic JSON parsing
- ✓ Better error handling
- ✓ Request/Response interceptors

axios

6 PERFORMANCE OPTIMIZATION

- ✓ Use React.memo to memoise components.
- ✓ Use useMemo for expensive calculations.
- ✓ Use useCallback for stable function refs.
- ✓ Avoid unnecessary re-renders.
- ✓ Profile with React DevTools.

Example

```

const MemoComp = React.memo((props) => {
  const value = useMemo(() => {
    expensiveCalc(props.data);
  }, [props.data]);

  const handleClick = useCallback(() => {
    doSomething(id);
  }, [id]);
});
  
```



7 ERROR BOUNDARY

```

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  render() {
    if (this.state.hasError) {
      return <div>Something went wrong </div>;
    }
    return this.props.children;
  }
}
  
```

Use

Wrap components that might crash. Shows fallback UI instead of breaking the whole app.



8 USEFUL SHIPPETS

Conditional Rendering

```
(isLoggedIn ? <Dashboard /> : <Login />)
```

List Rendering

```
(items.map(item => (
  <div key={item.id}>{item} </div>
)))
```

Fragment

```
<>
  <h1>Hello</h1>
  <p>World</p>
</>
```



9 BEST PRACTICES

- ✓ Keep components small & focused.
- ✓ Use meaningful names.
- ✓ Lift state up when needed.
- ✓ Avoid prop drilling (use Context).
- ✓ Follow single responsibility principle.
- ✓ Write clean, reusable & testable code.



10 COMMON PITFALLS

- ✗ Mutating state directly.
- ✗ Maxing dependency array in useEffect.
- ✗ Overusing effects.
- ✗ Inline functions causing re-renders.
- ✗ Not using keys in lists.
- ✗ Ignoring error handling.



11 DEV TOOLS

- React DevTools Inspect components & state.
- Profiler Find performance bottlenecks.
- Why Did You Render Detect unnecessary re-renders.

12 REMEMBER

- ✓ Think in Components.
- ✓ Props down, events up.
- ✓ Keep state minimal & local.
- ✓ Reuse logic with Hooks.
- ✓ Optimise wisely.
- ✓ Ship with confidence! 🚀

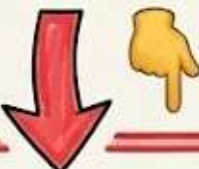


BONUS TIP: Practice daily. Build projects. Break things. For things. That's how you master React! ❤️

</> DevWithRishabh </>

READY TO MASTER REACT?

 Get the Complete PDF



**COMMENT
"REACT"
BELOW**

and I'll DM the complete PDF to you!

- ☒ Learn React Concepts (Beginner to Advanced)
- ☒ Revise Faster for Interviews
- ☒ Build Better Web Applications

DevWithRishabh

Learn • Build • Practice • Grow

Follow @DevWithRishabh for more practical developer notes & resources